Lesson 3 Teacher's Key

Unity

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution

Two scripts: Bullet.cs on the Bullet prefab, and an addition to the Player's existing script:

```
// Bullet.cs, attached to the Bullet prefab:
public float speed = 6f;

void Update() {
    transform.Translate(Vector3.up * speed * Time.deltaTime);
    if (transform.position.y > 6f) Destroy(gameObject);
}

// Player.cs - add a field and a check inside Update():
public GameObject bulletPrefab;

if (Keyboard.current.spaceKey.wasPressedThisFrame) {
    Instantiate(bulletPrefab, transform.position, Quaternion.identity);
}
```

Level 2 - Complete Worked Solution

wasPressedThisFrame is already edge-triggered (true for exactly one frame per physical press), so Level 1's fire check needs no changes for Level 2. An optional tunable cooldown, if a slower fire rate is wanted:

```
private float fireCooldown = 0f;

void Update() {
    fireCooldown -= Time.deltaTime;
    if (Keyboard.current.spaceKey.wasPressedThisFrame && fireCooldown <=
0f) {
        Instantiate(bulletPrefab, transform.position,
Quaternion.identity);
        fireCooldown = 0.2f;
    }
}
```

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking.

Sound: play on fire

Add an AudioSource component to the Player, AudioClip set to sfx_shoot.wav:

```
public AudioSource audioSrc;

if (Keyboard.current.spaceKey.wasPressedThisFrame && fireCooldown <= 0f) {
    Instantiate(bulletPrefab, transform.position, Quaternion.identity);
    fireCooldown = 0.2f;
    audioSrc.Play();
}
```

Visual: give the bullet its own look

Set color and scale in the Bullet script's own Start() so every spawned copy looks distinct without touching the Player script at all - then count a short flashTimer back down in Update() so the flash actually fades instead of staying permanent:

```
private float flashTimer;

void Start() {
    GetComponent<SpriteRenderer>().color = Color.yellow;
    transform.localScale *= 1.3f;
    flashTimer = 0.1f;
}

void Update() {
    if (flashTimer > 0f) {
        flashTimer -= Time.deltaTime;
        if (flashTimer <= 0f) {
            GetComponent<SpriteRenderer>().color = Color.white;
            transform.localScale /= 1.3f;
        }
    }
}
```

Structure: keep bullet behavior in Bullet.cs

Player.cs should only ever trigger a shot (Instantiate, cooldown, sound) - everything about what a bullet actually does once it exists (movement, off-screen cleanup, its own look) belongs in Bullet.cs. Keeping this boundary clean is what let the Sound and Visual ideas above each drop into one file without touching the other.

Complete Level 3 solution (all 3 goals combined)

Everything above combined, across Bullet.cs and Player.cs:

```
// Bullet.cs:
public float speed = 6f;
private float flashTimer;

void Start() {
    GetComponent<SpriteRenderer>().color = Color.yellow;
    transform.localScale *= 1.3f;
    flashTimer = 0.1f;
}

void Update() {
    transform.Translate(Vector3.up * speed * Time.deltaTime);
```

```
    if (transform.position.y > 6f) Destroy(gameObject);

    if (flashTimer > 0f) {
        flashTimer -= Time.deltaTime;
        if (flashTimer <= 0f) {
            GetComponent<SpriteRenderer>().color = Color.white;
            transform.localScale /= 1.3f;
        }
    }
}

// Player.cs:
public GameObject bulletPrefab;
public AudioSource audioSrc;
private float fireCooldown = 0f;

void Update() {
    fireCooldown -= Time.deltaTime;
    if (Keyboard.current.spaceKey.wasPressedThisFrame && fireCooldown <=
0f) {
        Instantiate(bulletPrefab, transform.position,
Quaternion.identity);
        fireCooldown = 0.2f;
        audioSrc.Play();
    }
}
```

Alternate Path Notes: Visual Scripting

The Fire action reuses the same PlayerControls Input Actions asset and Player Input component built in Lesson 2 - just one new Button action added to the existing Player action map, no new component setup required.

wasPressedThisFrame's single-frame-per-press behavior has a direct Script Graph equivalent: an On Input System Event Button node set to On Pressed only triggers once per physical press, so the same no-extra-cooldown-needed-for-Level-2 logic applies in graphs too.

The Level 3 goals map onto nodes the same way they map onto code: an Audio Source node for sound, "Set Member" nodes on the Sprite Renderer's color and the Transform's scale for the visual idea - all inside the Bullet prefab's own graph, kept separate from the Player's graph the same way Bullet.cs and Player.cs stay separate in the code path. The flash-then-fade behavior needs a Timer node (or a manually counted-down float variable checked each Update) so the same "Set Member" nodes fire a second time once the timer runs out, reverting color to white and dividing scale back down - without it, the color/scale change from Start stays permanent.

- Keeping the same Structure principle (bullet behavior lives with the bullet, not the player) whether a student is in C# or graphs reinforces that it's a design idea, not a syntax rule.
- Reusing Lesson 2's PlayerControls asset instead of building a second, separate Input Actions asset keeps every Input System-driven action for the whole capstone game consolidated in one place.